

CHƯƠNG 1. Cơ sở lý thuyết

1.1 Tổng quan

Chương 1 đã giới thiệu tổng quan về đề tài, xác định các vấn đề cần giải quyết và đề xuất định hướng giải pháp cho hệ thống "JobBridge AI". Chương này sẽ đi sâu vào việc trình bày các cơ sở lý thuyết, công nghệ và kiến trúc nền tảng đã được lựa chọn và áp dụng để xây dựng hệ thống. Việc hiểu rõ các khái niệm này là tiền đề quan trọng để theo dõi các phần phân tích, thiết kế và hiện thực hệ thống trong các chương tiếp theo.

Nội dung chính của chương bao gồm:

- Giới thiệu về kiến trúc Microservices và lý do lựa chọn.
- Trình bày về các công nghệ cốt lõi được sử dụng ở phía Backend (Go, Gin) và Frontend (React, TypeScript).
- Mô tả về công nghệ container hóa với Docker và nền tảng điều phối container với Kubernetes.
- Giới thiệu các khái niệm cơ bản về Trí tuệ nhân tạo, đặc biệt là các Mô hình ngôn ngữ lớn (LLMs) và cách ứng dụng chúng.

1.2 Kiến trúc Microservices

Kiến trúc microservices là một phương pháp phát triển phần mềm, trong đó một ứng dụng lớn được xây dựng dưới dạng một tập hợp các dịch vụ nhỏ, độc lập và có khả năng triển khai riêng lẻ. Mỗi dịch vụ thực thi một quy trình nghiệp vụ cụ thể và giao tiếp với các dịch vụ khác thông qua các giao diện lập trình ứng dụng (API) được định nghĩa rõ ràng, thường là qua giao thức HTTP.

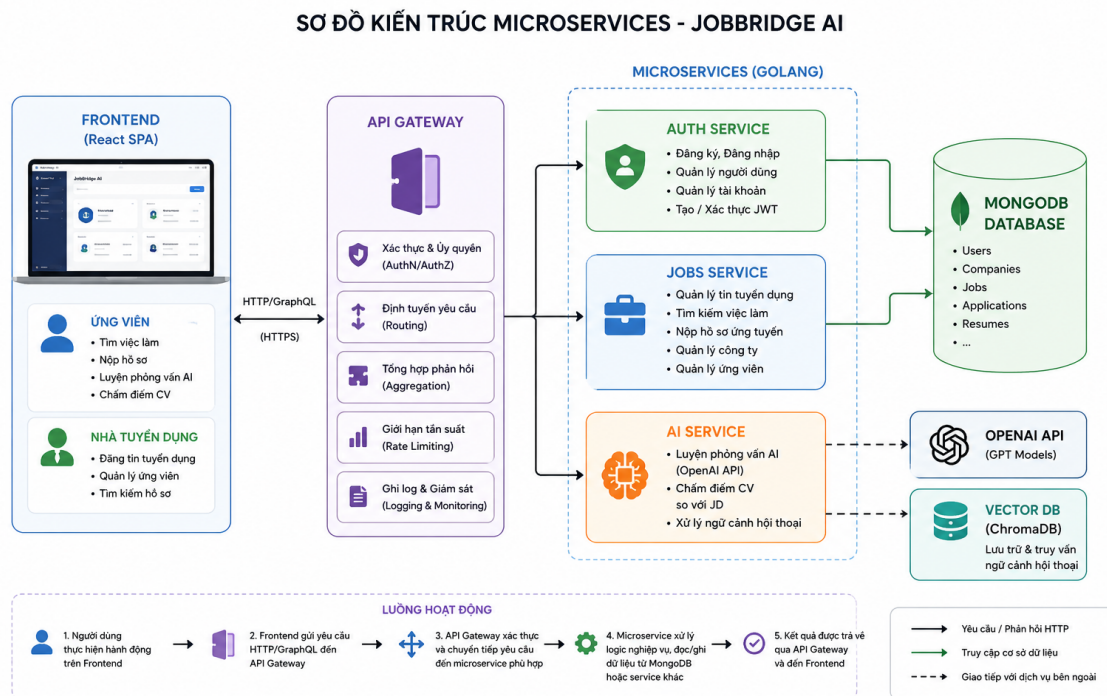
Việc lựa chọn kiến trúc microservices cho dự án "JobBridge AI" xuất phát từ những ưu điểm vượt trội mà nó mang lại so với kiến trúc nguyên khối (monolithic) truyền thống, đặc biệt phù hợp với một hệ thống phức tạp và có định hướng phát triển lâu dài:

- **Tính linh hoạt và khả năng mở rộng:** Mỗi microservice có thể được phát triển, triển khai và mở rộng quy mô một cách độc lập mà không ảnh hưởng đến các thành phần khác của hệ thống. Ví dụ, khi dịch vụ AI cần nhiều tài nguyên xử lý hơn, chúng ta có thể tăng số lượng bản sao (replicas) của chỉ dịch vụ đó.
- **Lựa chọn công nghệ đa dạng:** Mặc dù trong dự án này, tất cả các service đều được viết bằng Go, kiến trúc microservices cho phép mỗi dịch vụ có thể được

xây dựng bằng một công nghệ khác nhau, tùy thuộc vào yêu cầu cụ thể của từng bài toán.

- **Tăng cường khả năng bảo trì và phát triển:** Các dịch vụ nhỏ hơn, tập trung vào một nghiệp vụ duy nhất sẽ dễ hiểu, dễ phát triển và bảo trì hơn. Các nhóm phát triển khác nhau có thể làm việc song song trên các dịch vụ khác nhau, giúp đẩy nhanh tiến độ dự án.
- **Nâng cao khả năng chịu lỗi:** Nếu một dịch vụ gặp sự cố, nó sẽ không làm sập toàn bộ hệ thống. Các dịch vụ khác vẫn có thể tiếp tục hoạt động bình thường.

Như được minh họa trong Hình 1.1, kiến trúc của "JobBridge AI" bao gồm các thành phần chính sau:



Hình 1.1: Sơ đồ kiến trúc Microservices của JobBridge AI

- **Frontend (React SPA):** Là điểm tương tác duy nhất của người dùng. Tất cả các yêu cầu nghiệp vụ từ phía client sẽ được gửi đến API Gateway.
- **API Gateway:** Đóng vai trò là một lớp trung gian, là cổng vào duy nhất cho hệ thống backend. Nó chịu trách nhiệm xác thực, định tuyến yêu cầu đến các microservice phù hợp, giới hạn tần suất yêu cầu (rate limiting) và tổng hợp dữ liệu trả về cho client.
- **Các Microservices (Go):**
 - **Auth Service:** Chịu trách nhiệm toàn bộ về việc xác thực và quản lý người

dùng (đăng ký, đăng nhập, quản lý tài khoản, cấp phát và xác thực token JWT).

- **Jobs Service:** Xử lý các nghiệp vụ liên quan đến việc làm như quản lý tin tuyển dụng, tìm kiếm, và quy trình ứng tuyển.
- **AI Service:** Là bộ não của hệ thống, thực hiện các tác vụ thông minh như luyện tập phỏng vấn và chấm điểm CV bằng cách tương tác với các dịch vụ bên ngoài như OpenAI API.

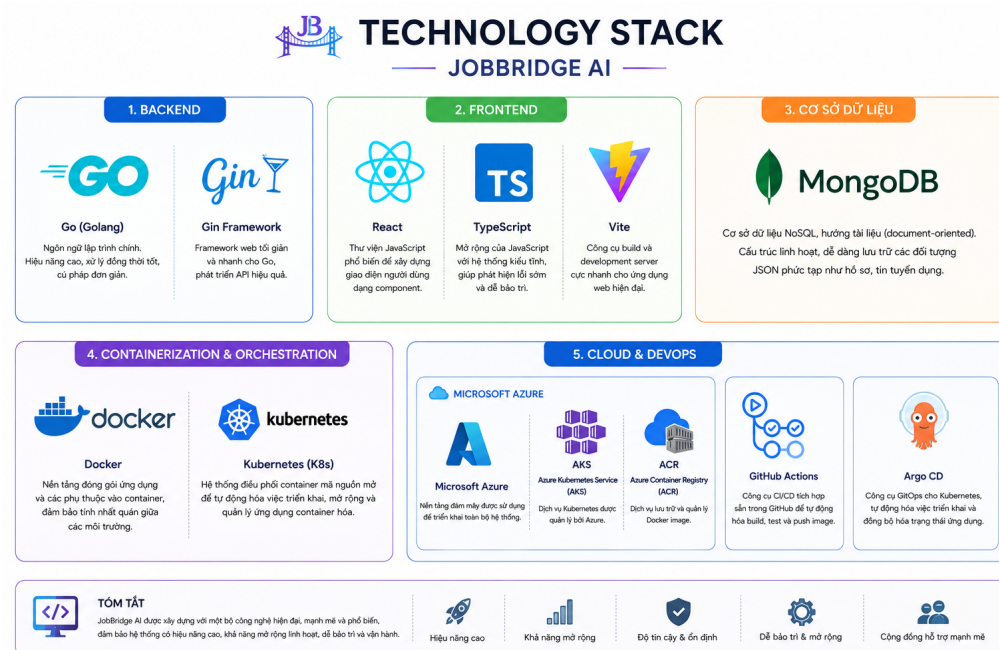
- **Cơ sở dữ liệu:**

- **MongoDB:** Được sử dụng làm cơ sở dữ liệu chính cho Auth Service và Jobs Service để lưu trữ dữ liệu người dùng, công ty, việc làm, và hồ sơ.
- **Vector DB (ChromaDB):** Được đưa vào trong thiết kế kiến trúc mục tiêu để lưu trữ và truy vấn ngữ cảnh hội thoại, giúp tính năng phỏng vấn thử trở nên thông minh và liền mạch hơn.

Luồng hoạt động cơ bản của hệ thống diễn ra theo trình tự: người dùng tương tác trên Frontend, yêu cầu được gửi đến API Gateway, Gateway xác thực và chuyển tiếp đến microservice tương ứng để xử lý, và cuối cùng kết quả được trả về cho người dùng. Mô hình này đảm bảo sự phân tách rõ ràng về trách nhiệm, giúp hệ thống trở nên linh hoạt và dễ dàng quản lý.

1.3 Các công nghệ sử dụng

Việc lựa chọn một bộ công nghệ (technology stack) phù hợp là yếu tố then chốt quyết định đến hiệu năng, khả năng mở rộng và bảo trì của một hệ thống phần mềm. Đối với "JobBridge AI", các công nghệ được lựa chọn đều là những công nghệ hiện đại, có cộng đồng phát triển lớn và đã được chứng minh hiệu quả trong các hệ thống quy mô lớn. Hình 1.2 cung cấp một cái nhìn tổng quan về các công nghệ được sử dụng trong dự án.



Hình 1.2: Các công nghệ chính được sử dụng trong dự án

1.3.1 Công nghệ phía Backend

Phía backend của hệ thống được xây dựng hoàn toàn bằng ngôn ngữ Go, kết hợp với các công nghệ hỗ trợ để đảm bảo hiệu suất và sự ổn định.

a, Ngôn ngữ Go (Golang)

Go là một ngôn ngữ lập trình biên dịch, mã nguồn mở được phát triển bởi Google. Go được lựa chọn làm ngôn ngữ chính cho việc xây dựng các microservice của "JobBridge AI" vì những lý do sau:

- **Hiệu năng cao:** Là một ngôn ngữ biên dịch, Go cung cấp hiệu suất gần tương đương với các ngôn ngữ như C++ hay Java, rất phù hợp cho các tác vụ xử lý yêu cầu nhanh và hiệu quả.
- **Hỗ trợ đồng thời (Concurrency) mạnh mẽ:** Go có cơ chế hỗ trợ lập trình đồng thời ngay từ trong nhân của ngôn ngữ thông qua Goroutines và Channels. Điều này giúp việc xây dựng các ứng dụng có khả năng xử lý hàng nghìn yêu cầu cùng lúc trở nên đơn giản và hiệu quả hơn rất nhiều so với các cơ chế đa luồng truyền thống.
- **Tối ưu cho ứng dụng mạng và microservice:** Go có một thư viện chuẩn mạnh mẽ, đặc biệt là trong lĩnh vực ứng dụng mạng. Cú pháp đơn giản, thời gian biên dịch nhanh và việc tạo ra một file thực thi duy nhất không có phụ thuộc ngoài giúp cho việc đóng gói và triển khai trong các container trở nên cực kỳ dễ dàng.

b, Gin Web Framework

Gin là một framework phát triển web hiệu suất cao được viết bằng Go. Nó cung cấp các tính năng cần thiết để xây dựng API một cách nhanh chóng nhưng vẫn giữ được hiệu năng tốt. Các đặc điểm nổi bật của Gin bao gồm hệ thống định tuyến (routing) nhanh, khả năng mở rộng thông qua các middleware (ví dụ: logging, xác thực), và khả năng xử lý lỗi tốt.

c, Cơ sở dữ liệu MongoDB

MongoDB là một hệ quản trị cơ sở dữ liệu NoSQL, hướng tài liệu (document-oriented). Thay vì lưu dữ liệu trong các bảng với các hàng và cột như cơ sở dữ liệu quan hệ, MongoDB lưu trữ dữ liệu dưới dạng các tài liệu BSON (một dạng nhị phân của JSON). Lựa chọn này mang lại nhiều lợi ích cho dự án:

- **Cấu trúc linh hoạt (Flexible Schema):** Rất phù hợp để lưu trữ dữ liệu có cấu trúc đa dạng và có thể thay đổi theo thời gian như hồ sơ ứng viên (CVs) hay mô tả công việc (JDs).
- **Khả năng mở rộng ngang:** MongoDB được thiết kế để có thể mở rộng quy mô một cách dễ dàng bằng cách thêm nhiều máy chủ vào cluster.
- **Hiệu năng truy vấn tốt:** Hỗ trợ các chỉ mục (indexes) mạnh mẽ và cung cấp một ngôn ngữ truy vấn phong phú cho các tài liệu JSON.

1.3.2 Công nghệ phía Frontend

Giao diện người dùng của "JobBridge AI" được xây dựng dưới dạng một ứng dụng trang đơn (Single Page Application - SPA) sử dụng các công nghệ hiện đại nhất trong hệ sinh thái JavaScript.

a, React và TypeScript

- **React:** Là một thư viện JavaScript do Facebook phát triển, được sử dụng để xây dựng giao diện người dùng. React cho phép chia nhỏ giao diện thành các thành phần (components) độc lập và có thể tái sử dụng, giúp quản lý trạng thái ứng dụng một cách hiệu quả và làm cho mã nguồn trở nên có tổ chức, dễ bảo trì.
- **TypeScript:** Là một ngôn ngữ lập trình được xây dựng trên nền tảng JavaScript, bổ sung thêm hệ thống kiểu tĩnh (static type system). Việc sử dụng TypeScript giúp phát hiện sớm các lỗi tiềm ẩn ngay trong quá trình phát triển, cải thiện chất lượng mã nguồn và tăng cường khả năng hỗ trợ từ các công cụ lập trình (code completion, refactoring).

b, Vite

Vite là một công cụ xây dựng (build tool) và máy chủ phát triển (dev server) thế hệ mới. So với các công cụ truyền thống như Webpack, Vite cung cấp một trải nghiệm phát triển vượt trội nhờ vào:

- **Khởi động máy chủ cực nhanh:** Tận dụng cơ chế native ES modules của trình duyệt, Vite có thể khởi động máy chủ phát triển gần như ngay lập tức.
- **Hot Module Replacement (HMR) siêu tốc:** Khi có sự thay đổi trong mã nguồn, Vite có thể cập nhật các thay đổi đó trên trình duyệt mà không cần tải lại toàn bộ trang, giúp chu trình phát triển diễn ra nhanh hơn đáng kể.

1.4 Nền tảng triển khai và DevOps

DevOps là một tập hợp các phương pháp luận kết hợp giữa phát triển phần mềm (Development) và vận hành công nghệ thông tin (Operations) nhằm mục đích rút ngắn vòng đời phát triển hệ thống và cung cấp các bản cập nhật, tính năng và sửa lỗi một cách liên tục, đồng thời đảm bảo chất lượng của phần mềm. Dự án "JobBridge AI" áp dụng triệt để các nguyên tắc DevOps thông qua việc sử dụng các công nghệ container hóa, điều phối và tự động hóa quy trình CI/CD.

1.4.1 Container hóa với Docker

Docker là một nền tảng mã nguồn mở cho phép tự động hóa việc triển khai ứng dụng bên trong các "container". Một container là một đơn vị phần mềm nhẹ, độc lập, có thể thực thi, bao gồm mọi thứ cần thiết để chạy một ứng dụng: mã nguồn, runtime, các công cụ hệ thống, và thư viện.

Trong dự án "JobBridge AI", mỗi microservice (Auth, Jobs, AI, Gateway) và cả ứng dụng Frontend đều được đóng gói vào một Docker image riêng biệt thông qua một file 'Dockerfile'. Việc sử dụng Docker mang lại các lợi ích cốt lõi:

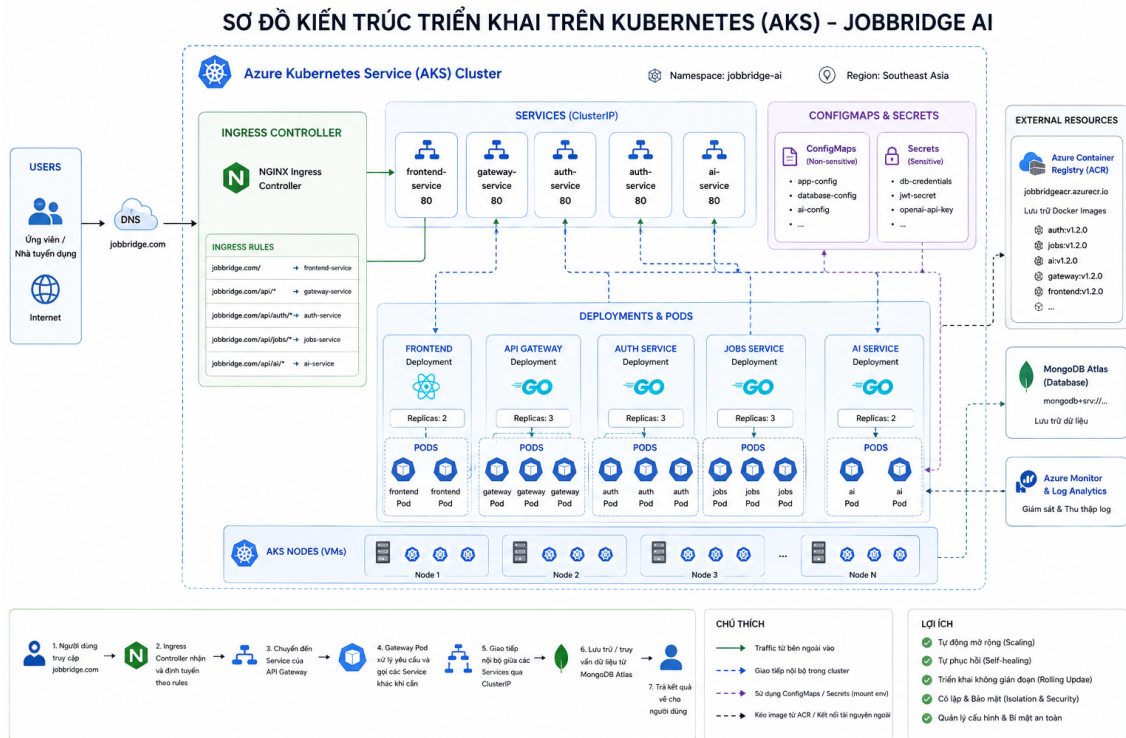
- **Tính nhất quán môi trường:** Đảm bảo rằng ứng dụng sẽ chạy theo cùng một cách, bất kể nó được triển khai ở đâu, từ máy tính cá nhân của lập trình viên cho đến môi trường production trên đám mây. Điều này giúp loại bỏ hoàn toàn vấn đề "It works on my machine".
- **Tính di động:** Docker container có thể chạy trên bất kỳ hệ thống nào có cài đặt Docker, từ Windows, macOS, Linux cho đến các nền tảng đám mây.
- **Tách biệt và bảo mật:** Các container chạy độc lập với nhau và với hệ điều hành chủ, giúp tăng cường tính bảo mật và ổn định.
- **Hiệu quả và nhẹ nhàng:** So với máy ảo (Virtual Machine), container chia sẻ nhân (kernel) của hệ điều hành chủ, giúp chúng khởi động nhanh hơn và tiêu

thụ ít tài nguyên hơn đáng kể.

1.4.2 Điều phối với Kubernetes

Khi một ứng dụng bao gồm nhiều container (như kiến trúc microservices), việc quản lý, kết nối và mở rộng quy mô chúng một cách thủ công sẽ trở nên vô cùng phức tạp. Kubernetes (thường được viết tắt là K8s) là một hệ thống điều phối container mã nguồn mở, được thiết kế để tự động hóa việc triển khai, mở rộng và vận hành các ứng dụng container hóa.

Dự án sử dụng Azure Kubernetes Service (AKS), một dịch vụ Kubernetes được quản lý hoàn toàn bởi Microsoft Azure, giúp đơn giản hóa việc triển khai và quản lý cluster. Các khái niệm chính của Kubernetes được áp dụng trong dự án được mô tả trong Hình 1.3:



Hình 1.3: Kiến trúc triển khai trên Azure Kubernetes Service

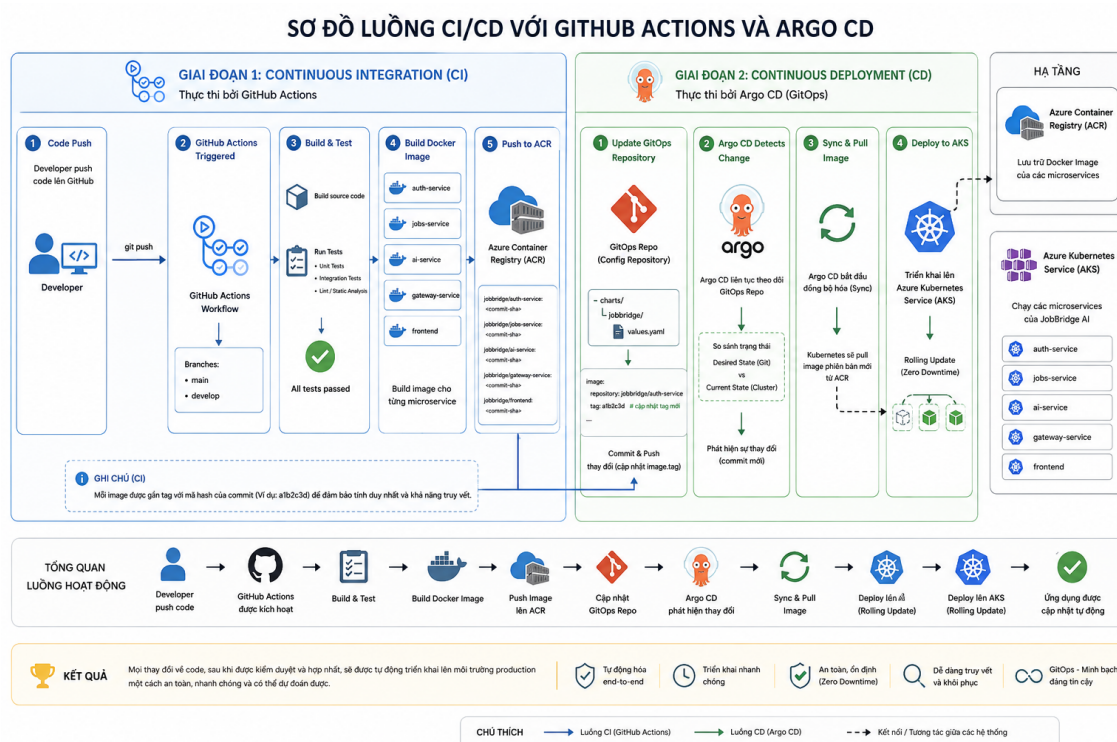
- **Pod:** Là đơn vị nhỏ nhất có thể triển khai trong Kubernetes, chứa một hoặc nhiều container. Trong dự án này, mỗi Pod sẽ chạy một container của một microservice.
- **Deployment:** Là một tài nguyên của Kubernetes dùng để quản lý các Pod. Nó đảm bảo rằng một số lượng bản sao (replicas) nhất định của một Pod luôn ở trạng thái sẵn sàng, đồng thời quản lý quá trình cập nhật phiên bản (rolling update) một cách an toàn.
- **Service:** Cung cấp một điểm truy cập mạng ổn định (một địa chỉ IP và DNS

name cố định) cho một nhóm các Pod. Điều này cho phép các microservice giao tiếp với nhau một cách dễ dàng bên trong cluster.

- **Ingress:** Quản lý truy cập từ bên ngoài vào các Service trong cluster, thường là các yêu cầu HTTP/HTTPS. Nó hoạt động như một bộ định tuyến thông minh, hướng lưu lượng truy cập đến đúng dịch vụ dựa trên tên miền hoặc đường dẫn.

1.4.3 CI/CD với GitHub Actions và Argo CD

Quy trình Tích hợp liên tục (CI) và Triển khai liên tục (CD) là trái tim của thực hành DevOps, giúp tự động hóa toàn bộ quá trình từ việc viết mã cho đến khi sản phẩm đến tay người dùng. Dự án áp dụng một quy trình CI/CD hiện đại dựa trên GitOps, được minh họa trong Hình 1.4.



Hình 1.4: Sơ đồ luồng CI/CD của dự án

a, Tích hợp liên tục (CI) với GitHub Actions

Quy trình CI được kích hoạt mỗi khi có mã nguồn mới được đẩy lên repository trên GitHub.

1. **Build và Test:** GitHub Actions tự động biên dịch mã nguồn và chạy các bộ kiểm thử (unit test) để đảm bảo chất lượng và tính đúng đắn của các thay đổi.
2. **Build Docker Image:** Nếu tất cả các bài kiểm thử đều thành công, quy trình sẽ tiến hành xây dựng các Docker image cho từng microservice.
3. **Push to Azure Container Registry (ACR):** Các image sau khi được build sẽ

được đẩy lên và lưu trữ tại ACR, một dịch vụ registry container riêng tư trên Azure.

b, Triển khai liên tục (CD) theo mô hình GitOps với Argo CD

GitOps là một phương pháp thực hành CD, trong đó một repository Git (GitOps repo) được coi là "nguồn chân lý" (source of truth) duy nhất cho việc định nghĩa trạng thái mong muốn của hệ thống.

1. **Update GitOps Repo:** Sau khi CI thành công, một bước tự động sẽ cập nhật file cấu hình (ví dụ: file 'values.yaml' của Helm) trong một repository Git thứ hai (GitOps repo), trở đến phiên bản image mới nhất vừa được đẩy lên ACR.
2. **Argo CD:** Là một công cụ GitOps chạy trong cluster Kubernetes. Nó liên tục theo dõi GitOps repo.
3. **Sync and Deploy:** Khi phát hiện có sự thay đổi trong GitOps repo, Argo CD sẽ tự động so sánh trạng thái mong muốn trong repo với trạng thái hiện tại trên cluster. Nếu có sự khác biệt, nó sẽ tự động "đồng bộ" (sync) bằng cách ra lệnh cho Kubernetes kéo image mới về và triển khai phiên bản cập nhật của ứng dụng.

Mô hình này đảm bảo rằng tất cả các thay đổi đối với môi trường vận hành đều được theo dõi, có thể kiểm tra và dễ dàng phục hồi thông qua lịch sử của Git.

1.5 Trí tuệ nhân tạo và Mô hình ngôn ngữ lớn (LLM)

Trí tuệ nhân tạo (Artificial Intelligence - AI) là một lĩnh vực rộng lớn của khoa học máy tính, tập trung vào việc tạo ra các hệ thống thông minh có khả năng thực hiện các tác vụ mà thông thường đòi hỏi trí thông minh của con người. Trong những năm gần đây, một nhánh của AI đã có những bước phát triển đột phá và thu hút sự chú ý đặc biệt, đó là các Mô hình ngôn ngữ lớn (Large Language Models - LLMs).

a, Mô hình ngôn ngữ lớn (LLM)

LLMs là các mô hình AI được huấn luyện trên một khối lượng dữ liệu văn bản khổng lồ, giúp chúng có được khả năng hiểu và tạo ra ngôn ngữ tự nhiên của con người một cách đáng kinh ngạc. Các mô hình này, chẳng hạn như dòng mô hình GPT (Generative Pre-trained Transformer) của OpenAI, có thể thực hiện một loạt các tác vụ ngôn ngữ phức tạp mà không cần được huấn luyện lại một cách cụ thể cho từng tác vụ. Các khả năng chính của chúng bao gồm:

- Trả lời câu hỏi.
- Tóm tắt văn bản.
- Dịch thuật.

- Viết văn, làm thơ, soạn email.
- Phân tích và đánh giá nội dung.
- Viết mã lập trình.

Kiến trúc nền tảng của hầu hết các LLM hiện đại là kiến trúc Transformer, cho phép mô hình xử lý các chuỗi dữ liệu (như câu chữ) và nắm bắt được mối quan hệ ngữ cảnh giữa các từ, kể cả khi chúng ở xa nhau trong một đoạn văn.

b, Ứng dụng LLM trong JobBridge AI

Dự án "JobBridge AI" không xây dựng một mô hình LLM từ đầu, mà tận dụng sức mạnh của các mô hình đã được huấn luyện sẵn thông qua các giao diện lập trình ứng dụng (API), cụ thể là OpenAI API. Cách tiếp cận này cho phép dự án tích hợp các khả năng AI tiên tiến một cách nhanh chóng và hiệu quả mà không đòi hỏi nguồn lực tính toán khổng lồ cho việc huấn luyện mô hình.

- **Trong tính năng luyện tập phỏng vấn:** Hệ thống gửi câu hỏi phỏng vấn và câu trả lời của người dùng đến LLM. Với vai trò là một chuyên gia tuyển dụng, LLM sẽ phân tích câu trả lời dựa trên các tiêu chí như sự liên quan, độ rõ ràng, cách diễn đạt và đưa ra những phản hồi mang tính xây dựng, giúp người dùng cải thiện kỹ năng của mình.
- **Trong tính năng chấm điểm CV:** Hệ thống trích xuất nội dung văn bản từ file CV của ứng viên và mô tả công việc (JD) từ nhà tuyển dụng. Cả hai nội dung này sẽ được đưa vào LLM. Mô hình sẽ thực hiện việc so sánh, đối chiếu kinh nghiệm, kỹ năng trong CV với các yêu cầu trong JD để đưa ra một điểm số đánh giá mức độ phù hợp và một bản tóm tắt ngắn gọn về điểm mạnh, điểm yếu của ứng viên.

Bằng cách này, "JobBridge AI" đã biến một công nghệ AI phức tạp thành một công cụ hữu ích và dễ tiếp cận, trực tiếp giải quyết các bài toán thực tế trong quy trình tuyển dụng.

1.6 Kết chương

Chương này đã trình bày một cách có hệ thống các cơ sở lý thuyết và công nghệ nền tảng của dự án "JobBridge AI". Từ kiến trúc Microservices, các công nghệ cụ thể cho backend và frontend, cho đến nền tảng triển khai DevOps hiện đại và các khái niệm về AI, tất cả đều là những mảnh ghép quan trọng tạo nên hệ thống. Những kiến thức này sẽ là cơ sở để đi vào chương tiếp theo, Chương 3, nơi sẽ trình bày chi tiết về việc phân tích và thiết kế hệ thống dựa trên các nền tảng đã chọn.